Mylo Wierez – Théo Freihuber – Marylou Dumas – Alexis Andries

Projet Transversal — DEVELOPPEMENT D'UNE APPLICATION OBJET A L'AIDE DU LANGAGE PYTHON

1. Introduction

Ce projet consiste en l'élaboration d'une application de gestion de ressources universitaires. Nous avons choisi de permettre à des étudiants et des professeurs d'interagir avec le portail d'une bibliothèque en ligne pour leur établissement.

2. Organisation du Programme

Le projet utilise la **Programmation Orientée Objet (POO)** pour structurer les données. L'objectif est de transformer de simples listes en objets intelligents qui portent leur propre logique.

La classe Ressource : C'est le modèle de base pour chaque livre. Elle contient son identifiant, son nom et son état de disponibilité.

```
class Ressource:
    def __init__(self, id_res,  nom, type_res, disponible=True):
        self.id_res = id_res
        self.nom = nom
        self.type_res = type_res
        self.disponible = disponible
```

Explication : Cette classe permet de créer des paternes pour chaque livre. La variable disponible est cruciale pour autoriser ou non un emprunt dès la première fois

3. Gestions Utilisateurs

L'application doit gérer des utilisateurs avec des droits différents. Tous les utilisateurs doivent être enregistre a la main dans le fichier, c'est le cas dans tous les établissement, les codes seront attribués à chaque élève et professeur pour ne pas pouvoir s'inscrire manuellement pour éviter d'avoir de multiple compte.

La classe Utilisateur : Elle gère les limites d'emprunts basées sur le rôle (Étudiant ou Professeur).

```
class Utilisateur:
    def __init__(self, identifiant, mdp, role, emprunts=None):
        self.identifiant = identifiant
        self.role = role
        self.emprunts = emprunts if emprunts else []
        # Règle : 5 pour profs, 3 pour étudiants
        self.max_emprunts = 5 if self.role == "prof" else 3

    def peut_emprunter(self):
        return len(self.emprunts) < self.max_emprunts
```

Explication : Dans cette méthode nous avons posé les règles que nous voulions en limitant le nombre de retrais ainsi qu'en faisant les vérifications nécessaires.

4. Sauvegarde des Données

Pour que les modifications ne soient pas perdues après la fermeture, les données sont stockées dans un fichier en .JSON. Il faut penser à importer au début du fichier ce dont l'on a besoin pour utiliser les fichiers à plat.

Le BibliothequeManager : C'est ce fichier qui nous sert de base de données

```
import json
import os

… (suite du code)

def sauvegarder_donnees(self):
    data = {
        "ressources": [r.to_dict() for r in self.ressources],
        "utilisateurs": [u.to_dict() for u in self.utilisateurs.values()]
    }
    with open("donnees.json", "w") as f:
        json.dump(data, f)
```

Explication : Cette fonction retranscrit toute les actions avec les données sous format JSON. Elle est appelée après chaque action importante (emprunt, retour ou ajout).

5. Interface Graphique

L'interface graphique (GUI) permet une interaction simple.

- **Sécurité** : Un écran de connexion vérifie l'identité de l'élève ou du professeur.
- **Interface dynamique** : Le bouton "Ajouter une ressource" est masqué pour les étudiants.
- **Gestion des erreurs** : Des boîtes de dialogue informent l'utilisateur si un ID est erroné ou si le livre est indisponible.